

CodeAgent Pro

File-by-file technical audit — architecture, control flow, sandboxing, RAG, evaluation, and risk surface

Prepared for: interview defense / design review

Scope: full repository (~~backend/~~, ~~frontend/~~, ~~docker/~~, ~~.github/~~)

Method: every claim below is anchored to a specific file and line range.

Repo state: branch `main`, HEAD `ce114fa` + post-audit revision (see Revision note). Line numbers refer to the revised working tree.

Revision note

This edition reflects a round of fixes applied after the first audit. Six items from §8 have been resolved in code; the rest are retained with an explicit **Defence** paragraph describing why they are acceptable for the current scope and how they would be closed.

#	Change	Files touched
1	Error-memory writes are now verified : a debugger fix is staged in <code>state["pending_fix"]</code> and only committed to ChromaDB after the next <code>run_tests</code> confirms <code>tests_passed</code> .	<code>core/nodes.py</code> , <code>core/state.py</code>
2	Local (unsandboxed) execution is now opt-in : without Docker and without <code>ALLOW_LOCAL_SANDBOX=true</code> , the sandbox raises <code>LocalSandboxDisabledError</code> and the graph aborts to <code>finalise</code> instead of running code on the host.	<code>core/sandbox.py</code> , <code>core/nodes.py</code> , <code>core/graph.py</code> , <code>backend/.env.example</code>
3	<code>/api/rag/index-project</code> is confined to the project root ; paths resolving outside it are rejected.	<code>api/routes.py</code>
4	<code>langgraph==1.2.2</code> pinned (was an undeclared hard dependency); <code>langchain</code> pin corrected to the installed <code>1.3.2</code> .	<code>backend/requirements.txt</code>
5	Dead state field <code>debug_context</code> removed. <code>max_debug_attempts</code> is now actually read by <code>after_tests_run</code> , so the eval <code>--max-debug</code> flag changes the real loop bound.	<code>core/state.py</code> , <code>core/graph.py</code>
6	CI runs the eval with <code>--compare</code> against <code>eval/runs/baseline.json</code> when present (failing the check on regression), and sets <code>ALLOW_LOCAL_SANDBOX=true</code> for the daemon-less runner. Latent <code>python -m eval.run_eval</code> working-directory bug fixed.	<code>.github/workflows/eval.yml</code>

Contents

- 1. Architecture map & LangGraph state machine
 - 1.1 The compiled graph — nodes and edges
 - 1.2 Conditional-edge functions
 - 1.3 PipelineState field-by-field
 - 1.4 Streaming & state persistence
- 2. Secure multi-language sandboxing
 - 2.1 `check_docker()`
 - 2.2 `run_code()`
 - 2.3 `run_tests()` & zero-test detection
 - 2.4 Container configuration matrix
- 3. Agent roles & the self-repair debug loop
 - 3.1 Prompt strategy per role
 - 3.2 Output parsing & fallback
 - 3.3 Full debug-loop trace
 - 3.4 Error memory persistence
- 4. Multi-collection RAG architecture & error memory
 - 4.1 Base vector store
 - 4.2 The three collections
 - 4.3 `get_generation_context()` vs `get_debug_context()`
- 5. Multi-provider LLM abstraction & routing
 - 5.1 Provider prefix parsing & auto-detection
 - 5.2 Dispatch, timeouts, 429 back-off
 - 5.3 `/models` health reporting
- 6. Offline benchmark & evaluation engine
 - 6.1 Harness
 - 6.2 Metrics, in plain English
 - 6.3 Comparator / regression gate
 - 6.4 Dataset & CI
- 7. API layer & frontend streaming architecture
 - 7.1 `/generate/stream` and SSE headers
 - 7.2 Frontend stream consumption & UI state
- 8. Gaps and weak points
 - 8A Resolved in this revision
 - 8B Deferred — with defence & path-to-close
- 9. One-page mental model — the request lifecycle

1. Architecture map & LangGraph state machine

CodeAgent Pro is a FastAPI backend that compiles a single LangGraph state graph once at import time and streams its execution to a React SPA over Server-Sent Events. The graph is a linear pipeline with one cycle (the debug loop) and four conditional "abort / branch" points. Nothing about a run is persisted; the run is the lifetime of one HTTP response.

1.1 The compiled graph — nodes and edges

The graph is assembled in `build_graph()` and compiled into a module-level singleton. Node *functions* live in `core/nodes.py`; `graph.py` only wires them together.

backend/core/graph.py:74-122 - `build_graph()`

```
def build_graph() -> StateGraph:
    g = StateGraph(PipelineState)

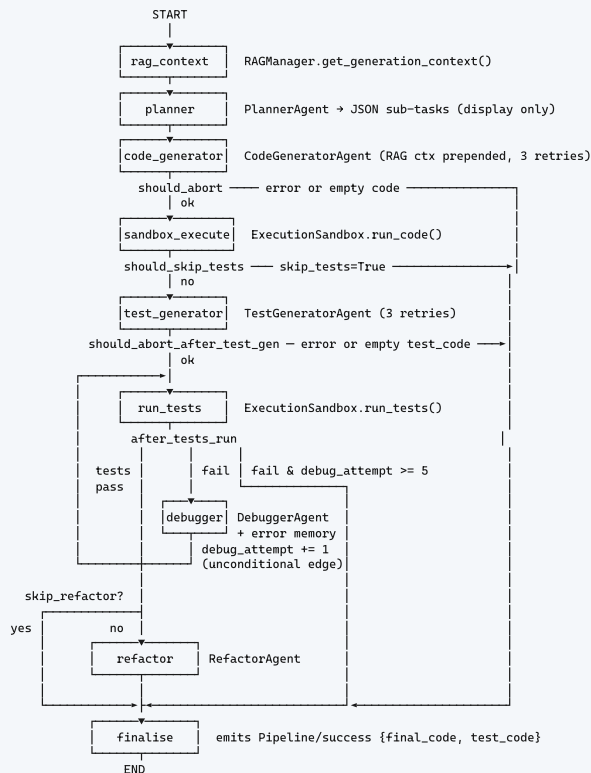
    g.add_node("rag_context", node_rag_context)
    g.add_node("planner", node_planner)
    g.add_node("code_generator", node_code_generator)
    g.add_node("sandbox_execute", node_sandbox_execute)
    g.add_node("test_generator", node_test_generator)
    g.add_node("run_tests", node_run_tests)
    g.add_node("debugger", node_debugger)
    g.add_node("refactor", node_refactor)
    g.add_node("finalise", node_finalise)

    g.set_entry_point("rag_context")
    g.add_edge("rag_context", "planner")
    g.add_edge("planner", "code_generator")
    g.add_edge("debugger", "run_tests") # debug loop closes here
    g.add_edge("refactor", "finalise")
    g.add_edge("finalise", END)

    g.add_conditional_edges("code_generator", should_abort,
        {"sandbox_execute": "sandbox_execute", "finalise": "finalise"})
    g.add_conditional_edges("sandbox_execute", should_skip_tests,
        {"test_generator": "test_generator", "finalise": "finalise"})
    g.add_conditional_edges("test_generator", should_abort_after_test_gen,
        {"run_tests": "run_tests", "finalise": "finalise"})
    g.add_conditional_edges("run_tests", after_tests_run,
        {"refactor": "refactor", "debugger": "debugger", "finalise": "finalise"})

    return g.compile()

agent_graph = build_graph() # singleton, imported by pipeline.py and eval/harness.py
```



Node	Function (nodes.py)	Reads from state	Writes to state
rag_context	node_rag_context :34	task, language, events	rag_context, events
planner	node_planner :45	task, model, events	plan, events
code_generator	node_code_generator :55	task, language, framework, rag_context, model	current_code or error, events
sandbox_execute	node_sandbox_execute :77	current_code, language	exec_stdout, exec_stderr, exec_success, events
test_generator	node_test_generator :104	current_code, task, language, model	test_code or error, events
run_tests	node_run_tests :124	current_code, test_code, language, debug_attempt	tests_passed, last_error, exec_stdout/stderr, events
debugger	node_debugger :168	last_error, current_code, debug_attempt, task, language, model	current_code, debug_attempt, events ; side-effect: error-memory write
refactor	node_refactor :204	current_code, language, model	final_code, current_code, events
finalise	node_finalise :221	final_code, current_code, test_code	final_code, events

Transition triggers. Every linear edge fires unconditionally the instant a node returns its partial-state dict. The only cycle is `debugger` → `run_tests` (a plain `add_edge`, line 92) paired with the `run_tests` → `debugger` branch inside `after_tests_run`. The loop counter is `debug_attempt`, incremented only by `node_debugger` (`attempt = state.get("debug_attempt", 0) + 1`, nodes.py:169). So the sequence is: `run_tests` (attempt 0) fails → `debugger` sets 1 → `run_tests` fails → `debugger` sets 2 → ... → `debugger` sets 5 → `run_tests` fails → `after_tests_run` sees `5 >= 5` → `finalise`. Maximum five debugger invocations per run.

1.2 Conditional-edge functions

backend/core/graph.py:32-69 – the four routers

```

MAX_DEBUG_ATTEMPTS = 5

def should_abort(state) -> str:
    # after code_generator
    if state.get("error") or not state.get("current_code"):
        return "finalise"
    return "sandbox_execute"

def should_skip_tests(state) -> str:
    # after sandbox_execute
    # e.g. sandbox disabled – abort
    if state.get("error"):
        return "finalise"
    if state.get("skip_tests"):
        return "finalise"
    return "test_generator"

def should_abort_after_test_gen(state) -> str:
    # after test_generator
    if state.get("error") or not state.get("test_code"):
        return "finalise"
    return "run_tests"

def after_tests_run(state) -> str:
    # after run_tests
    if state.get("tests_passed"):
        if state.get("skip_refactor"):
            return "finalise"
        return "refactor"
    attempts = state.get("debug_attempt", 0)
    limit = state.get("max_debug_attempts") or MAX_DEBUG_ATTEMPTS
    if attempts >= limit:
        return "finalise" # give up
    return "debugger"

```

Router	Source node	Branch condition → target
should_abort	code_generator	error set OR current_code falsy → finalise ; else → sandbox_execute
should_skip_tests	sandbox_execute	error set (e.g. no sandbox available) → finalise ; skip_tests truthy → finalise ; else → test_generator . The error check also stops the run if the initial execution set a fatal error. A non-fatal run failure (crash, wrong output) still proceeds — it is the test run that gates the debug loop.
should_abort_after_test_gen	test_generator	error set OR test_code empty → finalise ; else → run_tests
after_tests_run	run_tests	tests_passed → (skip_refactor ? finalise : refactor) ; elif debug_attempt >= (state max_debug_attempts or 5) → finalise ; else → debugger

5 appears as the default in three places — `graph.py:32` (`MAX_DEBUG_ATTEMPTS`, now the *fallback* when state carries no bound), `pipeline.py:18` (`AgentPipeline.MAX_DEBUG_ATTEMPTS`, written into the initial state as `max_debug_attempts`), and `DebuggerAgent(max_attempts=5)` (`debugger.py:12`, used only for the "attempt N of 5" prompt text). Since the fix, `after_tests_run` reads `state["max_debug_attempts"]` first, so the eval harness's `--max-debug` value now actually changes the loop bound.

1.3 PipelineState field-by-field

backend/core/state.py:9-45 – the shared state object

```

class PipelineState(TypedDict):
    # Input
    task: str
    language: str
    framework: str
    model: str
    skip_tests: bool
    skip_refactor: bool
    # Planner
    plan: list[dict]          # [{id, title, description, files}]
    # RAG context
    rag_context: str
    # Code generation
    current_code: str
    test_code: str
    # Execution
    exec_stdout: str
    exec_stderr: str
    exec_success: bool
    # Debug loop
    debug_attempt: int
    max_debug_attempts: int  # loop bound, read by after_tests_run
    tests_passed: bool
    last_error: str
    pending_fix: Optional[dict]  # staged fix; persisted only after verification
    # Refactor
    final_code: str
    # Event log (streamed to frontend)
    events: list[dict]
    error: Optional[str]

```

`PipelineState` is a bare `TypedDict` — no `Annotated[...]`, `reducer` on any field. `LangGraph` therefore treats every field as last-write-wins: when a node returns `{"events": [...]}`, that value *replaces* the channel. To emulate an append log, every node reconstructs the full list by hand, e.g. `events = state["events"] + [_event(...)]` (nodes.py, throughout). The `_event` helper (nodes.py:28) just builds `{"step", "status", "data"}`.

Field	Purpose	Written by	Read by
<code>task</code>	the natural-language request	initial state	<code>rag_context</code> , <code>planner</code> , <code>code_generator</code> , <code>debugger</code> (recorded), <code>harness</code>
<code>language</code>	Python / JavaScript / C++	initial state	<code>rag_context</code> , <code>code_generator</code> , both <code>sandbox</code> nodes, <code>test_generator</code> , <code>debugger</code> , <code>refactor</code>
<code>framework</code>	free-text hint injected into the codegen prompt	initial state	<code>code_generator</code> only
<code>model</code>	provider-prefixed model id, e.g. <code>groq:openai/gpt-oss-20b</code>	initial state	<code>_llm(state)</code> helper (nodes.py:24) for every LLM node
<code>skip_tests</code>	UI toggle: stop after the first run	initial state	<code>should_skip_tests</code>
<code>skip_refactor</code>	UI toggle: finalise instead of polishing	initial state	<code>after_tests_run</code>
<code>plan</code>	ordered sub-task list from the planner	<code>node_planner</code>	<i>nothing downstream</i> — surfaced only in the streamed event, does not drive generation
<code>rag_context</code>	docs + codebase snippets block	<code>node_rag_context</code>	<code>node_code_generator</code> (prepended to the prompt)
<code>pending_fix</code>	a staged debugger fix — <code>{error, broken_code, fixed_code, task, language}</code> — awaiting verification	<code>node_debugger</code> (sets it on a successful call), <code>node_run_tests</code> (clears it)	<code>node_run_tests</code> : if the run passes, the pending fix is committed to error memory, then cleared
<code>current_code</code>	the working solution, mutated in place across the loop	<code>code_generator</code> , <code>debugger</code> , <code>refactor</code>	<code>sandbox_execute</code> , <code>test_generator</code> , <code>run_tests</code> , <code>debugger</code> , <code>refactor</code> , <code>finalise</code>
<code>test_code</code>	generated test suite	<code>test_generator</code>	<code>run_tests</code> , <code>finalise</code>
<code>exec_stdout</code> / <code>exec_stderr</code> / <code>exec_success</code>	result of the initial <code>run_code</code> , later overwritten by <code>run_tests</code>	<code>sandbox_execute</code> , <code>run_tests</code>	no graph logic reads these; only streamed to the UI's "Exec Output" tab
<code>debug_attempt</code>	self-repair iteration counter	<code>node_debugger</code>	<code>after_tests_run</code> , <code>node_run_tests</code> (label only), <code>harness</code>
<code>max_debug_attempts</code>	the debug-loop bound	initial state (<code>AgentPipeline.MAX_DEBUG_ATTEMPTS</code> , or the eval <code>--max-debug</code> value)	<code>after_tests_run</code> — <code>state.get("max_debug_attempts")</code> or <code>MAX_DEBUG_ATTEMPTS</code> ; also the eval harness
<code>tests_passed</code>	true only if the suite ran <i>and</i> passed <i>and</i> collected > 0 tests	<code>node_run_tests</code>	<code>after_tests_run</code> , <code>harness</code>
<code>last_error</code>	<code>stdout</code> + <code>"\n"</code> + <code>stderr</code> of the failing test run	<code>node_run_tests</code>	<code>node_debugger</code> (fed to the debugger prompt)
<code>final_code</code>	the deliverable	<code>refactor</code> , <code>finalise</code>	<code>finalise</code> , <code>harness</code> , <code>pipeline</code>
<code>events</code>	append-only log of <code>{step, status, data}</code>	every node (manual concat)	<code>AgentPipeline.run</code> streaming loop
<code>error</code>	fatal-failure marker for the abort routers	<code>node_code_generator</code> , <code>node_test_generator</code> (on failure)	<code>should_abort</code> , <code>should_abort_after_test_gen</code> ; never cleared once set

1.4 How `AgentPipeline` streams snapshots as SSE, and what persists

backend/core/pipeline.py:36-81 – the async adapter

```

initial_state: PipelineState = {
    "task": task, "language": language, "framework": framework,
    "model": self.model, "skip_tests": skip_tests, "skip_refactor": skip_refactor,
    "plan": [], "rag_context": "", "debug_context": "",
    "current_code": "", "test_code": "",
    "exec_stdout": "", "exec_stderr": "", "exec_success": False,
    "debug_attempt": 0, "max_debug_attempts": self.MAX_DEBUG_ATTEMPTS,
    "tests_passed": False, "last_error": "", "final_code": "",
    "events": [], "error": None,
}
seen_event_count = 0
try:
    async for state_snapshot in agent_graph.astream(initial_state):
        for node_name, node_state in state_snapshot.items():
            events = node_state.get("events", [])
            new_events = events[seen_event_count:]
            for event in new_events:
                yield event
            seen_event_count = len(events)
except Exception as e:
    yield {"step": "Pipeline", "status": "failed",
          "data": {"error": f"Pipeline crashed: {str(e)}"}}
yield {"step": "Done", "status": "success", "data": {}}
```

Mechanics. `agent_graph.astream(initial_state)` runs with `LangGraph`'s default `stream_mode="updates"` , so each iteration yields one `{node_name: partial_state}` dict — the delta that node returned. Because every node returns the *entire* reconstructed `events` list, `pipeline.py` can keep a high-water mark

(`seen_event_count`), slice off only the newly appended events, and yield each one. The API layer (Section 7) wraps each yielded dict as `data: {json}\n\n`. A crash anywhere in the graph is caught here and turned into a single synthetic `Pipeline/failed` event; a sentinel `Done/success` always terminates the generator.

State persistence: there is none **RISK**

`build_graph()` calls `g.compile()` with **no checkpointer argument**. There is no `thread_id`, no SQLite/Postgres saver, no run identifier. The entire run lives in three ephemeral places: the local `initial_state` dict, LangGraph's in-process channel values, and the open HTTP response. Consequences of a backend restart mid-execution:

- The `uvicorn` process is started with `reload=True` (`main.py:40`), so *any file save* during development kills in-flight runs.
- The SSE socket drops. On the client, `useAgent.run`'s `fetch` reader throws; the hook sets `status: 'failed'` with the network error. There is no resume path — the user re-submits from scratch.
- Nothing about the run is recoverable: no partial code, no event history, no "which node were we on".
- What *does* survive: ChromaDB writes. Error-memory rows are now written only *after* a test run verifies the fix (revision fix #1), so a crash mid-loop no longer poisons memory with an unverified fix — but a fix that was verified earlier in the same run is already committed and persists to `backend/data/chromadb/` even though the run produced no deliverable.

Defence & roadmap. For the current target — a single-user local dev tool — "the run dies, the user re-submits" is a tolerable failure mode, and the verified-write change removes the one way a dead run could corrupt state. To make runs durable: compile with a checkpointer (`MemorySaver` for in-process resume; `SQLiteSaver` / `PostgresSaver` for cross-restart), assign each run a `thread_id`, return it in the first SSE event, and add a `GET /generate/stream/{run_id}` that replays the event log and re-attaches to the live graph via `astream(..., config={"configurable": {"thread_id": run_id}})`. That is ~15 lines at the `compile()` / endpoint boundary and does not touch node logic.

2. Secure multi-language sandboxing

`core/sandbox.py` is 187 lines with one class, `ExecutionSandbox`, plus a module-level Docker probe. It has two public methods — `run_code` (compile/run once) and `run_tests` (run a suite) — and both choose between a Docker container and a bare-subprocess "local fallback". The fallback is automatic and near-silent.

2.1 `check_docker()` — the probe

backend/core/sandbox.py:8–34

```
_DOCKER_AVAILABLE = None
FORCE_LOCAL_SANDBOX = os.getenv("FORCE_LOCAL_SANDBOX", "false").lower() == "true"

async def check_docker() -> bool:
    global _DOCKER_AVAILABLE
    if _DOCKER_AVAILABLE is not None:
        return _DOCKER_AVAILABLE # memoised for process lifetime
    if not shutil.which("docker"):
        _DOCKER_AVAILABLE = False; return False
    try:
        import subprocess
        res = await asyncio.to_thread(subprocess.run, ["docker", "info"],
            stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=2)
        _DOCKER_AVAILABLE = (res.returncode == 0)
    except Exception:
        _DOCKER_AVAILABLE = False
    return _DOCKER_AVAILABLE
```

Two gates: `docker` must be on `PATH`, and `docker info` must exit 0 within 2 seconds (run in a worker thread so the event loop is never blocked). The result is cached in the module global `_DOCKER_AVAILABLE` for the life of the process — **starting Docker Desktop after the backend booted has no effect until a restart**. `FORCE_LOCAL_SANDBOX=true` short-circuits the whole thing (used by `test_sandbox.py` and available for CI).

2.2 `run_code()` — container vs local, per language

backend/core/sandbox.py:46–89

```
async def run_code(self, code: str, language: str = "Python") -> dict:
    use_docker = (not FORCE_LOCAL_SANDBOX) and (await check_docker())
    lang = language.lower()
    ext = ".py"
    if "javascript" in lang or "js" in lang: ext = ".js"
    elif "c++" in lang or "cpp" in lang: ext = ".cpp"
    filename = f"solution{ext}"

    with tempfile.TemporaryDirectory() as tmpdir:
        (Path(tmpdir) / filename).write_text(code, encoding="utf-8")
        if use_docker:
            if ext == ".js":
                cmd = ["docker", "run", "--rm", "--network", "none", "--memory", "256m",
                    "-v", f"{tmpdir}:/workspace:ro", "-w", "/workspace",
                    "node:20-alpine", "node", filename]
            elif ext == ".cpp":
                cmd = ["docker", "run", "--rm", "--network", "none", "--memory", "256m",
                    "-v", f"{tmpdir}:/workspace:ro", "-w", "/workspace",
                    "gcc:latest", "sh", "-c", f"g++ -O2 {filename} -o solution && ./solution"]
            else:
                cmd = ["docker", "run", "--rm", "--network", "none", "--memory", "256m",
                    "-v", f"{tmpdir}:/workspace:ro", "-w", "/workspace",
                    "python:3.11-slim", "python", filename]
            return await self._run_subprocess(cmd)
        else:
            # local fallback (opt-in)
            if not _local_allowed(): raise LocalSandboxDisabledError(_LOCAL_DISABLED_MSG)
            if ext == ".js": cmd = [shutil.which("node") or "node", filename]
            elif ext == ".cpp":
                exe = "solution.exe" if sys.platform == "win32" else "./solution"
                build = await self._run_subprocess(["g++", "-O2", filename, "-o", "solution"], cwd=tmpdir)
                if not build["success"]:
                    build["stderr"] = ["WARNING: Running in local fallback mode"] + build["stderr"]
                return build
            cmd = [str(Path(tmpdir)/exe)]
        else:
            cmd = [sys.executable, filename]
        res = await self._run_subprocess(cmd, cwd=tmpdir)
        res["stderr"] = ["WARNING: Running in local fallback mode without Docker sandbox"] + res["stderr"]
    return res
```

Isolation model (Docker path). Source is written to a host `TemporaryDirectory` and bind-mounted into `/workspace`. Every container gets `--rm` (no leftover container), `--network none` (no egress, no DNS, no localhost), and `--memory 256m` (OOM-kill above the cap). Python and JS mount the workspace **read-only** (`:ro`) because they only need to read one file; C++ mounts **read-write** because `g++` must emit the `solution` binary next to the source. Wall-clock is bounded by `_run_subprocess`'s default `TIMEOUT_SECONDS = 30`. There is no `--cpus`, no `--pids-limit`, no `--read-only` root FS, no `seccomp/apparmor` profile, no non-root `--user` — so within the 256 MB / 30 s / no-network envelope the code runs as root inside the container.

Local fallback is now opt-in **FIXED**

Previously, when `check_docker()` was false the "else" branch ran the model's code with `sys.executable / node / g++` directly on the host — automatically, with only a `stderr` warning string that nothing inspected. Since revision fix #2:

```
FORCE_LOCAL_SANDBOX = os.getenv("FORCE_LOCAL_SANDBOX", "false") ...
ALLOW_LOCAL_SANDBOX = os.getenv("ALLOW_LOCAL_SANDBOX", "false").lower() == "true"

class LocalSandboxDisabledError(RuntimeError): ...

def _local_allowed() -> bool:
    return ALLOW_LOCAL_SANDBOX or FORCE_LOCAL_SANDBOX

# inside run_code() / run_tests(), the else (non-Docker) branch:
if not _local_allowed():
    raise LocalSandboxDisabledError(_LOCAL_DISABLED_MSG)
```

`node_sandbox_execute` and `node_run_tests` catch `LocalSandboxDisabledError`, set `state["error"]`, and emit a `Sandbox/failed` event; `should_skip_tests` then routes straight to `finalise`. So the default behaviour with no Docker is a clean abort with an actionable message, not silent host execution. Running untrusted code on the host now requires an explicit, documented opt-in (`ALLOW_LOCAL_SANDBOX=true`), which CI sets deliberately because its runner has no Docker daemon.

2.3 `run_tests()` — the three test runners, and zero-test detection

backend/core/sandbox.py:91-135 (Docker branch shown)

```
(Path(tmpdir) / f"solution{ext}").write_text(code, ...)
(Path(tmpdir) / f"test_solution{test_ext}").write_text(test_code, ...)
# test_ext: .py -> .py | .js -> .test.js | .cpp -> _test.cpp

if ext == ".js":
    cmd = ["docker", "run", "--rm", "--network", "none", "--memory", "512m",
          "-v", f"{tmpdir}:/workspace", "-w", "/workspace",
          "node:20-alpine", "node", "--test", f"test_solution{test_ext}"]
elif ext == ".cpp":
    cmd = [..., "--memory", "512m", "-v", f"{tmpdir}:/workspace", "gcc:latest", "sh", "-c",
          f"g++ -O2 solution.cpp test_solution_test.cpp -o test_runner && ./test_runner"]
else: # Python
    cmd = [..., "--memory", "512m", "-v", f"{tmpdir}:/workspace", "python:3.11-slim", "sh", "-c",
          "pip install pytest -q 2>/dev/null && pytest test_solution.py -v --tb=short 2>&1"]
return await self._run_subprocess(cmd, timeout=60)
```

- **Python:** `pytest test_solution.py -v --tb=short`. The test file does `from solution import <symbol>`. Memory is raised to 512 MB and timeout to 60 s (vs 256 MB / 30 s for a plain run). The workspace is mounted read-write so `pytest` can create `.pytest_cache`.
- **JavaScript:** Node's built-in runner, `node --test test_solution.test.js`. Suites use `require('node:test') + require('node:assert')` and pull the symbol out of `./solution.js`.
- **C++:** compile the solution and the test translation unit together (`g++ -O2 solution.cpp test_solution_test.cpp -o test_runner`) and run the binary. The "suite" is an `int main()` full of `assert(...)` that prints "All tests passed!" and returns 0.

The Dockerised Python test path cannot work as written **SHARP EDGE**

`python:3.11-slim` does not ship `pytest`, and the command tries to `pip install pytest` inside a container started with `--network none`. The install fails, `&&` short-circuits, `pytest` never runs, and `sh -c` exits non-zero. In practice Python tests only succeed via the *local fallback* branch (`sys.executable -m pytest`), which works because the backend's own `virtualenv` has `pytest==8.2.2` (`requirements.txt`). Every committed evaluation run in `backend/eval/runs/` carries the "[WARNING: Running in local fallback mode]" marker, confirming the Docker test path is not the one being exercised.

Defence & roadmap. Not fixed in this revision because it needs an image build, not a code change. The correct fix is a small `docker/Dockerfile.sandbox-python` that is `FROM python:3.11-slim + RUN pip install pytest`, built once and referenced by tag; the C++ and JS runners already work offline because `g++` and `node --test` ship in their base images. Until then, the Docker path is effectively Python-incomplete and the tool relies on the (now opt-in) local runner for Python test execution.

backend/core/nodes.py:124-153 - `node_run_tests`: zero-test guard

```
result = await ExecutionSandbox().run_tests(state["current_code"], state["test_code"],
                                           language=state.get("language", "Python"))
tests_passed = result["success"]
stdout_low = result.get("stdout", "").lower()
if "collected 0 items" in stdout_low or "no tests ran" in stdout_low:
    tests_passed = False
...
error_out = result["stdout"] + "\n" + result["stderr"]
return {"tests_passed": tests_passed, "last_error": error_out, ...}
```

Why the guard matters. If `TestGenerator` emits something that imports cleanly but defines no `test_*` functions, `pytest` exits 0 with "collected 0 items" on `stdout`. Without the check, `result["success"]` would be true and `after_tests_run` would route straight to `refactor` — a false "all tests passed". Forcing `tests_passed = False` on the two telltale phrases ("collected 0 items", "no tests ran") sends the run into the debug loop instead, where the debugger gets the raw `pytest` output as its error context. The same phrase check does not exist for the Node or C++ runners — those rely on the runner's own exit code.

2.4 `_run_subprocess` and the container matrix

backend/core/sandbox.py:137-179

```

async def _run_subprocess(self, cmd, timeout=None, cwd=None) -> dict:
    timeout = timeout or self.TIMEOUT_SECONDS # 30
    def run_sync():
        import subprocess
        try:
            res = subprocess.run(cmd, stdout=PIPE, stderr=PIPE, timeout=timeout, cwd=cwd)
            return {"stdout": res.stdout.decode("utf-8", "replace"),
                    "stderr": res.stderr.decode("utf-8", "replace"),
                    "exit_code": res.returncode, "success": res.returncode == 0,
                    "timed_out": False}
        except subprocess.TimeoutExpired as e:
            return {"stdout": "...", "stderr": "... or "Execution timed out",
                    "exit_code": -1, "success": False, "timed_out": True}
        except FileNotFoundError:
            raise FileNotFoundError
    try:
        return await asyncio.to_thread(run_sync)
    except FileNotFoundError:
        return {"stdout": "", "stderr": "Execution binary not found.",
                "exit_code": -1, "success": False, "timed_out": False}

```

Every external process — Docker or local — goes through this one function. It deliberately uses blocking `subprocess.run` inside `asyncio.to_thread` rather than `asyncio.create_subprocess_exec`, because Uvicorn's default event loop on Windows raises `NotImplementedError` for async subprocesses (this was a real crash that dropped the SSE stream; the thread-offload is the fix). The return contract is uniform: `{stdout, stderr, exit_code, success, timed_out}`. A missing binary surfaces as `exit_code -1 + "Execution binary not found."`; a timeout as `exit_code -1 + timed_out True`. The eval harness keys on exactly those markers to classify "sandbox error" vs "code wrong" (Section 6.1).

	Python	JavaScript	C++
Image	python:3.11-slim	node:20-alpine	gcc:latest
Run cmd	python solution.py	node solution.js	g++ -O2 solution.cpp -o solution && ./solution
Test cmd	pip install pytest && pytest -v --tb=short	node --test test_solution.test.js	g++ -O2 solution.cpp test_...test.cpp -o test_runner && ./test_runner
Network	--network none (all three, both methods)		
Memory	--memory 256m for run_code; --memory 512m for run_tests		
Mount	:ro (run) / rw (test)	:ro (run) / rw (test)	rw (needs to emit a binary)
Timeout	30 s (run_code) / 60 s (run_tests), enforced host-side by subprocess.run(timeout=...)		
Local fallback	sys.executable, -m pytest	node on PATH	g++ on PATH, solution.exe on Windows

3. Agent roles & the self-repair debug loop

Five agent classes wrap the LLM client. All prompt text is centralised in `core/prompts.py` as `*_SYSTEM` / `*_PROMPT` pairs. Four of the five (all but the planner) share an identical `_clean_code` extractor. Only `CodeGenerator` and `TestGenerator` retry internally; the debugger and refactorer get one shot per call.

3.1 Prompt-engineering strategy, per role

backend/core/prompts.py – system + user templates

```
CODE_GENERATOR_SYSTEM = """You are an expert software engineer.
Your job is to write clean, well-commented, production-ready code.
Always return ONLY the code – no explanations, no markdown fences unless asked.
If multiple files are needed, separate them with: ### FILE: <filename> ###"""

CODE_GENERATOR_PROMPT = """Write code for the following task:
TASK: {task}
LANGUAGE: {language}
FRAMEWORK: {framework}
CRITICAL: You MUST write the code strictly in {language}. Do NOT write code in
Python unless LANGUAGE is Python.
Requirements: error handling; docstrings/comments for {language}; runnable immediately.
Return only the code."""

DEBUGGER_PROMPT = """The following code has an error. Fix it.
ORIGINAL CODE:
{code}
ERROR / TEST OUTPUT:
{error}
ATTEMPT NUMBER: {attempt} of {max_attempts}
Rules: return the complete fixed file, not just changed lines; fix ALL issues in
the error; do not change working parts; maintain the original programming language.
Return only the fixed code."""

REFACTOR_PROMPT = """Refactor this working {language} code for quality and readability:
CODE:
{code}
Focus on: idiomatic {language}; remove dead code; better names; type hints if
applicable; simplify logic; DO NOT convert the code to another programming
language! Stay strictly in {language}.
Return only the refactored code."""

PLANNER_SYSTEM = """You are a technical lead who breaks down feature requests into
clear sub-tasks. Always respond in valid JSON only."""
PLANNER_PROMPT = """Break down this coding task into atomic sub-tasks. TASK: {task}
Return a JSON array like:
[{"id": 1, "title": "...", "description": "...", "files": ["models.py"]}]]
Return only valid JSON."""

TEST_GENERATOR_PROMPT = """Write comprehensive unit tests for the following code:
LANGUAGE: {language}
CODE: {code}
TASK DESCRIPTION: {task}
Requirements by language:
- Python: Use pytest. Import from 'solution.py'.
- JavaScript: Use Node.js assert/test runner. Require from './solution.js'.
- C++: Write a test file with int main(), #include <cassert>, #include "solution.cpp",
  run assertions, print "All tests passed!" and return 0 on success.
Return ONLY the test code."""
```

Agent	Prompt levers	Retry behaviour
Planner planner.py	Role = "technical lead". Hard constraint "valid JSON only" in <i>both</i> system and user. Few-shot: one literal JSON array example. Output is advisory — it is streamed to the UI but never fed to another node.	No LLM retry. Any exception → a one-item fallback plan (<code>[{"id":1, "title:task, ...}]</code>) with <code>success:False</code> , so the graph never stalls on planning.
CodeGenerator code_generator.py	Role = "expert software engineer". Repeated language lock ("CRITICAL... strictly in {language}"). Explicit "ONLY the code". RAG block is prepended as <code>RELEVANT CONTEXT: \n{context}\n\n</code> when present. <code>temperature 0.2</code> .	Up to <code>max_attempts=3</code> ; <code>await asyncio.sleep(2 * attempt)</code> between tries. Empty output counts as failure. After 3, returns <code>success:False</code> → <code>node_code_generator</code> sets <code>state["error"]</code> → <code>should_abort</code> → finalise.
TestGenerator test_generator.py	Role = "senior QA engineer". Per-language runner instructions baked into the user prompt (pytest import path / Node require path / C++ include+assert+banner). Passes the just-generated <code>current_code</code> verbatim.	Same 3-attempt / <code>sleep(2*attempt)</code> loop as CodeGenerator. Empty test code → failure → <code>should_abort_after_test_gen</code> → finalise.
Debugger debugger.py	Role = "expert debugger". Gets the full failing file, the concatenated <code>stdout+stderr</code> , and "ATTEMPT {n} of {max}". Rules forbid partial diffs and language switching. The error string is <i>augmented</i> with a past-fix block from error memory before the call (3.3).	No retry loop. On LLM exception it returns the <i>original</i> code with <code>success:False</code> — the loop then re-tests unchanged code and will burn the remaining attempts.
Refactor refactor.py	Role = "senior engineer / code quality". Language lock is the dominant instruction ("DO NOT convert... Stay strictly in {language}") — added after refactor was observed rewriting working C++/JS into Python. Runs only on code that already passed tests.	No retry. On exception, passes the input code through unchanged with <code>success:False</code> .

3.2 JSON parsing and the fence-stripping fallback

backend/agents/code_generator.py:63-80 – shared by 4 agents verbatim

```
def _clean_code(self, raw: str) -> str:
    if not raw: return ""
    import re
    text = raw.strip()
    pattern = r"```(?:[a-zA-Z0-9_+#+-])?\n(?:[\\s\\S]*?)\\n```"
    matches = list(re.finditer(pattern, text))
    if matches:
        largest = max(matches, key=lambda m: len(m.group(1)))
        return largest.group(1).strip()
    lines = text.splitlines()
    if lines and lines[0].startswith("```"): lines = lines[1:]
    if lines and lines[-1].strip() == "```": lines = lines[:-1]
    return "\n".join(lines).strip()
```

Two-tier extraction. Reasoning-tuned models (gpt-oss, qwen, Gemini) tend to wrap code in one or more ````lang ... ```` blocks preceded and followed by prose. Tier 1: find *all* fenced blocks and keep the one with the longest body — this discards short “here’s a snippet of the fix” asides and keeps the full file. Tier 2 (no fences at all): strip a leading ````` line and a trailing ````` line if present, else return the text as-is. The net effect is that conversational preambles never reach the sandbox or the Code viewer.

backend/agents/planner.py:38-49 – `_parse_plan` (planner only)

```
def _parse_plan(self, raw: str) -> list[dict]:
    cleaned = raw.strip()
    if cleaned.startswith("```"): cleaned = "\n".join(cleaned.splitlines()[1:])
    if cleaned.endswith("```"): cleaned = "\n".join(cleaned.splitlines()[:-1])
    parsed = json.loads(cleaned.strip())
    if not isinstance(parsed, list):
        raise ValueError("Expected a JSON array from planner")
    return parsed
```

The planner strips a single leading/trailing fence line, then `json.loads`. A non-list result raises. Any failure (bad JSON, wrong shape, LLM error) is caught in `PlannerAgent.run` and swallowed into the one-item fallback plan — planning cannot abort the pipeline.

3.3 The debug loop, traced end to end

backend/core/nodes.py – `node_debugger` (staging) + `node_run_tests` (verified commit)

```
async def node_debugger(state) -> dict:
    attempt = state.get("debug_attempt", 0) + 1
    events = state["events"] + [_event("Debugger", "running", {"attempt": attempt})]

    error = state.get("last_error", "")
    memory_ctx = (_rag.get_debug_context(error) if RAG_AVAILABLE else "")
    augmented_error = error + ("\n\n" + memory_ctx if memory_ctx else "")

    result = await DebuggerAgent(_llm(state)).run(state["current_code"], augmented_error, attempt)

    # Stage the fix; it is committed to error memory by node_run_tests only
    # after the next run confirms tests_passed.
    pending_fix = None
    if RAG_AVAILABLE and result["success"] and result.get("fixed_code"):
        pending_fix = {"error": error, "broken_code": state["current_code"],
                      "fixed_code": result["fixed_code"],
                      "task": state["task"], "language": state["language"]}

    return {"current_code": result["fixed_code"], "debug_attempt": attempt,
            "pending_fix": pending_fix, "events": events}

# and in node_run_tests, once tests_passed is known:
pending_fix = state.get("pending_fix")
if tests_passed and RAG_AVAILABLE and pending_fix:
    _rag.record_successful_fix(**pending_fix)
```

- run_tests** executes `solution + test_solution` in the sandbox. It sets `tests_passed` (with the zero-test override), and `last_error = stdout + "\n" + stderr` — the entire pytest/node/g++ transcript, stack trace included.
- after_tests_run**: if `tests_passed` → refactor (or finalise); if `debug_attempt >= 5` → finalise (give up); else → debugger.
- node_debugger**: `attempt = debug_attempt + 1`. It queries error memory with the raw error (`_get_debug_context`, 3.4), and if a past fix scores above threshold, *appends* that block to the error text before handing it to `DebuggerAgent`. The debugger returns a full rewritten file.
- Stage**: if the debugger call produced non-empty code, the tuple `(error, broken_code, fixed_code, task, language)` is placed in `state["pending_fix"]` — not yet written to the DB.
- Commit on verification**: the loop re-enters `node_run_tests`. If that run now reports `tests_passed`, `node_run_tests` calls `_rag.record_successful_fix(**pending_fix)` and clears the slot. If it still fails, `pending_fix` is cleared and the next `node_debugger` stages a fresh candidate. A fix is written to long-term memory only after a real test run confirms it (see 3.4).
- Loop**: `node_debugger` writes `current_code = fixed_code` and `debug_attempt = attempt`; the unconditional `debugger` → `run_tests` edge re-enters step 1.
- Exit**: on the first passing run → refactor; after the 5th debugger call fails its re-test → finalise with the last (still-broken) `current_code` as `final_code`. The UI shows `DebugLoop/failed "Max debug attempts reached (5)"`.

3.4 Error-memory persistence — `record_successful_fix()`

backend/rag/error_memory_store.py:7-27 — `ErrorMemoryStore.record_fix`

```
def record_fix(self, error, broken_code, fixed_code, task, language):
    doc = (f"Task: {task}\n"
          f"Language: {language}\n"
          f"Error Output:\n{error}\n\n"
          f"Broken Code:\n{broken_code}\n\n"
          f"Fixed Code:\n{fixed_code}")
    metadata = {"task": task, "language": language,
               "error_summary": error[:200].replace("\n", " ")}
    self.add([doc], [metadata], [f"fix_{uuid.uuid4().hex}"]) # embed + upsert
```

backend/rag/rag_manager.py:38-58 — retrieval + write wrappers

```
def get_debug_context(self, error_output: str) -> str:
    try:
        results = self.error_memory_store.query(error_output, n_results=1)
        if results and results[0]["score"] > 0.65:
            return f"\n### Found Past Fix for Similar Error:\n{results[0]['content']}\n"
        return ""
    except Exception:
        return ""

def record_successful_fix(self, error, broken_code, fixed_code, task, language):
    try:
        self.error_memory_store.record_fix(error, broken_code, fixed_code, task, language)
    except Exception:
        pass
```

Storage. Each fix is one ChromaDB document: a single string containing task, language, full error output, the broken file, and the fixed file, embedded with MiniLM (Section 4). Metadata carries a 200-char single-line error summary for quick scanning. IDs are random (`fix_{uuid4}`) so nothing is deduplicated.

Retrieval. On every debug attempt, the raw error string is embedded and matched against the `error_memory` collection, top-1. Only if cosine similarity (`score = 1 - distance`) exceeds **0.65** is the stored document injected, verbatim, under a `### Found Past Fix for Similar Error:` heading appended to the debugger's error input. Below threshold, nothing is injected. This is the mechanism intended to stop the debugger re-deriving a fix it already found on a previous run.

Error-memory writes are now verified FIXED

Previously `node_debugger` called `record_successful_fix` the moment `DebuggerAgent.run` returned without raising — true for *any* non-empty model response — so a wrong fix, or prose that slipped past `_clean_code`, could be embedded as a "successful fix" and retrieved later. That is the poisoning pattern that motivated the `/api/rag/clear` endpoint. Since revision fix #1 the write is deferred: `node_debugger` only *stages* the candidate in `state["pending_fix"]`, and `node_run_tests` performs the `record_successful_fix(**pending_fix)` call only on a run where `tests_passed` is true, then clears the slot. A malformed or ineffective fix is dropped when the following test run fails, and never reaches the vector store. `/api/rag/clear` is retained as an operator escape hatch, not a routine necessity.

4. Multi-collection RAG architecture & error memory

RAG is a thin layer over ChromaDB with local embeddings. One base class, three collections, one manager. Everything runs offline: the embedding model is `all-MiniLM-L6-v2` (384-dim) loaded once per process; the vector DB is a persistent on-disk Chroma at `backend/data/chromadb/`. There is no reranker, no external retrieval API, and no chunk-level metadata beyond file name / index.

4.1 Base vector store

backend/rag/base_store.py:10-83

```
DB_PATH = Path(__file__).parent.parent / "data" / "chromadb"

def get_chroma_client() -> chromadb.PersistentClient:
    DB_PATH.mkdir(parents=True, exist_ok=True)
    return chromadb.PersistentClient(path=str(DB_PATH),
        settings=Settings(anonymized_telemetry=False))

_EMBEDDING_MODEL = None
def _get_embedding_model():
    global _EMBEDDING_MODEL
    if _EMBEDDING_MODEL is None:
        from sentence_transformers import SentenceTransformer
        _EMBEDDING_MODEL = SentenceTransformer("all-MiniLM-L6-v2")
    return _EMBEDDING_MODEL

class BaseVectorStore:
    COLLECTION_NAME = "base"
    def __init__(self):
        self.client = get_chroma_client()
        self.collection = self.client.get_or_create_collection(
            name=self.COLLECTION_NAME, metadata={"hnsw:space": "cosine"})

    def _embed(self, texts): return _get_embedding_model().encode(texts, show_progress_bar=False).tolist()

    def add(self, documents, metadatas, ids):
        self.collection.upsert(documents=documents, embeddings=self._embed(documents),
            metadatas=metadatas, ids=ids)

    def query(self, query_text, n_results=5):
        results = self.collection.query(query_embeddings=self._embed([query_text]),
            n_results=n_results, include=["documents", "metadatas", "distances"])
        return [{"content": d, "metadata": m, "score": 1 - dist}
            for d, m, dist in zip(results["documents"][0], results["metadatas"][0], results["distances"][0])]
```

- **Similarity space:** each collection is created with `metadata={"hnsw:space": "cosine"}`, so Chroma's HNSW index uses cosine distance. `query()` converts to a similarity with `score = 1 - distance`, which is what the `> 0.65` gate in `get_debug_context` compares against.
- **Embedding model:** a module-global singleton. Before it existed, every store instance reloaded `SentenceTransformer` from disk on each query — 10–12 reloads per pipeline run, ~36 s of `rag_context` latency. It is now cached and also pre-warmed at server startup (`main.py`'s `@app.on_event("startup")` calls `_get_embedding_model()`), bringing retrieval to tens of milliseconds.
- **Persistence:** `PersistentClient` writes SQLite + HNSW segments under `backend/data/chromadb/`; collections survive restarts. `clear()` deletes documents by ID (chosen over `delete_collection` to dodge Windows SQLite file-locking).
- **Blocking:** `_embed` runs `model.encode` synchronously on the calling thread. The RAG nodes call it without `await` / `to_thread`, so embedding briefly blocks the event loop — acceptable for a single-user local tool, not for concurrency.

4.2 The three collections

Store	Collection	What goes in	How it's populated
<code>DocsStore</code> <code>docs_store.py</code>	<code>docs</code>	~55 hand-written reference snippets: FastAPI (routing, CORS, <code>StreamingResponse</code> , DI, background tasks, WebSockets, OAuth2, testing, lifespan...), pytest basics + <code>async</code> , <code>uvicorn</code> , <code>SQLAlchemy</code> , <code>Pydantic</code> , and a large DSA set (Big-O table, arrays/strings, two-pointer, sliding window, linked lists, stacks/queues/heaps, union-find, sorts, recursion/backtracking, trees/tries, graph BFS/DFS/Dijkstra/topo-sort, DP memo/tabulation/LCS, greedy).	<code>seed_builtins()</code> , IDs <code>doc_builtin_{i}</code> . Runs only when a user calls <code>POST /api/rag/seed-docs</code> (the "Seed built-in docs" button). Nothing seeds automatically on boot.
<code>CodebaseStore</code> <code>codebase_store.py</code>	<code>codebase</code>	Char-window chunks (<code>chunk_size=800</code> , <code>overlap=150</code>) of local source. Metadata: absolute POSIX <code>file_path</code> , <code>file_name</code> , <code>chunk_index</code> . IDs <code>code_{uuid12}_{idx}</code> .	<code>index_directory(path)</code> via <code>POST /api/rag/index-project {"directory": ...}</code> . The endpoint now resolves the path and rejects anything outside the project root (revision fix #3). <code>os.walk</code> with a prune list (<code>.venv</code> , <code>venv</code> , <code>.git</code> , <code>__pycache__</code> , <code>data</code> , <code>chromadb</code> , <code>node_modules</code> , <code>dist</code> , <code>build</code>). Indexes *.py only — JS/C++ project files are never picked up.
<code>ErrorMemoryStore</code> <code>error_memory_store.py</code>	<code>error_memory</code>	One doc per recorded fix: task + language + full error + broken file + fixed file (Section 3.4).	<code>record_fix(...)</code> , called by <code>node_debugger</code> during the loop. Grows monotonically until <code>POST /api/rag/clear</code> .

4.3 `get_generation_context()` vs `get_debug_context()`, step by step

backend/rag/rag_manager.py:11-36 – generation context

```
def get_generation_context(self, task: str, language: str = "Python") -> str:
    try:
        doc_results = self.docs_store.query(task, n_results=3)
        code_results = self.codebase_store.query(task, n_results=3)
        context_blocks = []
        if doc_results:
            context_blocks.append("### Relevant Documentation Reference:")
            for res in doc_results:
                context_blocks.append(f"- {res['content']}")
        if code_results:
            context_blocks.append("### Relevant Codebase Snippets:")
            for res in code_results:
                fn = res["metadata"].get("file_name", "unknown")
                context_blocks.append(f"File: {fn}\n```python\n{res['content']}\n```")
        return "\n\n".join(context_blocks) if context_blocks else ""
    except Exception:
        return ""
```

	<code>get_generation_context(task, language)</code>	<code>get_debug_context(error_output)</code>
Called by	<code>node_rag_context</code> (once, before codegen)	<code>node_debugger</code> (every debug attempt)
Collections queried	<code>docs</code> (top 3) + <code>codebase</code> (top 3)	<code>error_memory</code> (top 1)
Query text	the raw task string	the raw <code>stdout+stderr</code> transcript
Score threshold	none — the top 3 of each are taken unconditionally, however weak the match	score > 0.65 — below this, nothing is returned
Output shape	### Relevant Documentation Reference: + - {snippet} lines, then ### Relevant Codebase Snippets: + File: x\n```python ... ``` blocks	### Found Past Fix for Similar Error: + the full stored fix document
Where it lands in the prompt	Prepended by <code>CodeGeneratorAgent.run</code> as <code>RELEVANT CONTEXT:\n{ctx}\n\n{prompt}</code>	Appended to the error text: <code>augmented_error = error + "\n\n" + memory_ctx</code>
Language handling	parameter is accepted but unused; codebase snippets are always fenced as <code>```python```</code> regardless of target language	n/a
Failure mode	any exception → <code>""</code> (RAG is best-effort; codegen proceeds context-free)	any exception → <code>""</code>

Seeding is opt-in and manual. "FastAPI / Pytest / Python idioms / DSA" only enter the `docs` collection after an explicit `POST /api/rag/seed-docs`. "Local project directory indexing" only happens after `POST /api/rag/index-project` with a directory the caller supplies (the README example indexes `./backend`). On a fresh install with no seeding, `node_rag_context` still runs but `get_generation_context` returns `""` and code generation proceeds with no retrieved context. `RAGManager.stats()` (surfaced at `GET /api/rag/stats` and in the left-panel counters) reports `collection.count()` for each of the three.

Why RAG at all, given frontier models already know FastAPI?

The design's own answer (from the project's engineering log): pre-trained models don't know a team's *private* code, and single-shot generation can't learn from an execution failure. The `codebase` collection exists to ground generation in local imports and utilities; the `error_memory` collection is a growing, run-to-run memory of concrete bug fixes so the debugger doesn't repeat itself. The `docs` collection is essentially a curated cache to reduce dependence on the model's parametric knowledge for the framework surface this project actually uses.

5. Multi-provider LLM abstraction & routing

`core/llm_client.py` is one class, `OllamaClient` (aliased `LLMClient`), ~170 lines. It fronts three back ends — local Ollama, Groq's OpenAI-compatible API, and Google Gemini — behind a single `async generate(prompt, system) -> str`. Provider selection is by string prefix, with keyword auto-detection as a fallback.

5.1 Provider prefix parsing & auto-detection

```
backend/core/llm_client.py:18-42 - __init__

def __init__(self, model="deepseek-coder:6.7b", provider=None):
    self.raw_model = model
    self.provider = provider
    # explicit prefix, e.g. "groq:llama-3.3-70b-versatile"
    if ":" in model and not provider:
        parts = model.split(":", 1)
        if parts[0].lower() in ["ollama", "groq", "gemini"]:
            self.provider = parts[0].lower(); self.model = parts[1]
        else:
            self.model = model
    else:
        self.model = model
    if not self.provider:
        # keyword auto-detect
        if "gemini" in self.model.lower():
            self.provider = "gemini"
        elif any(k in self.model.lower() for k in
            ["groq", "gpt-oss", "qwen", "llama-3.3", "mixtral", "gemma2-9b"]):
            self.provider = "groq"
        else:
            self.provider = "ollama"
    self.base_url = OLLAMA_BASE_URL # env, default http://localhost:11434
```

Two resolution paths. (1) Explicit prefix: if the model string contains `:` and the part before it is one of `ollama|groq|gemini`, that wins and the remainder is the real model id. Note the API layer always constructs this form — `routes.generate_stream` builds `full_model = f"{req.provider}:{req.model}"`, so `"groq:openai/gpt-oss-20b"` parses to provider `groq`, model `openai/gpt-oss-20b`. The Ollama default `"deepseek-coder:6.7b"` has a `:` but `deepseek-coder` isn't a known provider, so the whole string is kept as the model and auto-detect falls through to `ollama`. (2) Auto-detect: substring checks on the lowercased model name — `gemini` → Gemini; any of a small keyword set (`gpt-oss`, `qwen`, `llama-3.3`, `mixtral`, `gemma2-9b`) → Groq; otherwise Ollama.

```
backend/core/llm_client.py:44-51 - dispatch

async def generate(self, prompt, system=""):
    if self.provider == "groq": return await self._generate_groq(prompt, system)
    elif self.provider == "gemini": return await self._generate_gemini(prompt, system)
    else: return await self._generate_ollama(prompt, system)
```

5.2 HTTP dispatch, timeouts, and 429 back-off

```
backend/core/llm_client.py:54-105 - Ollama and Groq back ends

async def _generate_ollama(self, prompt, system=""):
    payload = {"model": self.model, "prompt": prompt, "system": system,
        "stream": False,
        "options": {"temperature": 0.2, "top_p": 0.9, "num_predict": 4096}}
    async with httpx.AsyncClient(timeout=120.0) as client:
        r = await client.post(f"{self.base_url}/api/generate", json=payload)
        r.raise_for_status()
        return r.json().get("response", "")

async def _generate_groq(self, prompt, system=""):
    api_key = os.getenv("GROQ_API_KEY", "").strip()
    if not api_key: raise ValueError("GROQ_API_KEY is missing ...")
    url = "https://api.groq.com/openai/v1/chat/completions"
    headers = [{"Authorization": f"Bearer {api_key}"}, {"Content-Type": "application/json"}]
    messages = ([{"role": "system", "content": system}] if system else []) + \
        [{"role": "user", "content": prompt}]
    payload = {"model": self.model, "messages": messages,
        "temperature": 0.2, "max_tokens": 4096}
    for attempt in range(4):
        async with httpx.AsyncClient(timeout=120.0) as client:
            r = await client.post(url, headers=headers, json=payload)
            if r.status_code == 429 and attempt < 3:
                await asyncio.sleep(3 * (attempt + 1)) # 3s, 6s, 9s
                continue
            r.raise_for_status()
            return r.json()["choices"][0]["message"]["content"]
```

```
backend/core/llm_client.py:108-146 - Gemini back end
```



```

async def _generate_gemini(self, prompt, system=""):
    api_key = (os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY") or "").strip()
    if not api_key: raise ValueError("GEMINI_API_KEY is missing ...")
    model_name = self.model if self.model.startswith("models/") else f"models/{self.model}"
    url = f"https://generativelanguage.googleapis.com/v1beta/{model_name}:generateContent?key={api_key}"
    full_text = f"System Instruction: {system}\n\nUser Request: {prompt}" if system else prompt
    payload = {"contents": [{"parts": [{"text": full_text}]}],
              "generationConfig": {"temperature": 0.2, "maxOutputTokens": 4096}}
    for attempt in range(4):
        async with httpx.AsyncClient(timeout=120.0) as client:
            r = await client.post(url, json=payload)
            if r.status_code == 429 and attempt < 3:
                await asyncio.sleep(3 * (attempt + 1)); continue
            r.raise_for_status()
            cands = r.json().get("candidates", [])
            if not cands: raise ValueError("Gemini returned empty candidates ...")
            parts = cands[0].get("content", {}).get("parts", [])
            return "".join(p.get("text", "") for p in parts if isinstance(p, dict))

```

	Ollama	Groq	Gemini
Transport	<code>httpx.AsyncClient(timeout=120.0)</code> , a fresh client per call (and per retry attempt)		
Endpoint	<code>{OLLAMA_BASE_URL}/api/generate</code>	<code>api.groq.com/openai/v1/chat/completions</code>	<code>generativelanguage.googleapis.com/v1beta/{model}:generateContent</code>
Auth	none (local)	Authorization: Bearer <code>\$GROQ_API_KEY</code>	?key= <code>\$GEMINI_API_KEY</code> in the URL query string
System prompt	native <code>system</code> field	native <code>system</code> role message	folded into the user text as <code>"System Instruction: ..."</code>
Sampling	temperature 0.2, top_p 0.9, num_predict 4096	temperature 0.2, max_tokens 4096	temperature 0.2, maxOutputTokens 4096
429 handling	none — <code>raise_for_status</code> propagates	<code>for attempt in range(4); on 429 & attempt < 3, asyncio.sleep(3*(attempt+1)) = 3/6/9 s, then retry; 4th 429 propagates</code>	
Missing key	n/a	<code>ValueError</code> raised <i>inside the graph node</i> , surfaced as a <code>CodeGenerator/failed</code> event and an abort to <code>finalise</code>	

`import asyncio` at the top of the module is load-bearing: without it the `await asyncio.sleep(...)` in the retry loops raised `NameError`, which silently crashed `TestGenerator / Debugger` and let the run exhaust all five debug attempts. `load_dotenv()` is also called at module import here (not just in `main.py`) so the CLI eval path picks up `.env` regardless of entrypoint.

5.3 /models — how provider health is reported

backend/api/routes.py:42-73 — list_models()

```

@router.get("/models")
async def list_models():
    client = OllamaClient()
    ollama_ok = await client.is_available()
    ollama_models = await client.list_models() if ollama_ok else []
    groq_key = os.getenv("GROQ_API_KEY", "").strip()
    gemini_key = (os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY") or "").strip()
    return {
        "ollama": {"available": ollama_ok, "models": ollama_models},
        "groq": {"configured": bool(groq_key),
                "models": ["openai/gpt-oss-20b", "qwen/qwen3.6-27b", "openai/gpt-oss-120b"]},
        "gemini": {"configured": bool(gemini_key),
                  "models": ["gemini-3.6-flash", "gemini-3.7-flash", "gemini-3.5-flash"]},
    }

```

backend/core/llm_client.py:149-167 — the Ollama-only probes

```

async def list_models(self):
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            resp = await client.get(f"{self.base_url}/api/tags")
            if resp.status_code == 200:
                return [m["name"] for m in resp.json().get("models", [])]
    except Exception: pass
    return []

async def is_available(self):
    try:
        async with httpx.AsyncClient(timeout=3.0) as client:
            return (await client.get(f"{self.base_url}/api/tags")).status_code == 200
    except Exception:
        return False

```

Asymmetric verification. Ollama gets a live probe: `GET /api/tags` with a 3 s timeout for the up/down flag, and a 5 s `GET /api/tags` to enumerate installed models. Groq and Gemini get no network call at all — `configured` is simply `bool()` of a stripped environment variable, and the model lists are hard-coded constants. An invalid or expired API key therefore reports as "configured / online" in the header badge; the failure only surfaces when the run reaches `node_code_generator` and the `ValueError` / HTTP error aborts the pipeline. The frontend consumes this shape in `App.jsx` on mount to populate the provider badge and the model dropdown.

6. Offline benchmark & evaluation engine

`backend/eval/` runs the *same* compiled `agent_graph` against a fixed 18-task dataset, then executes a held-out acceptance test against the final code, aggregates metrics, writes a timestamped JSON, and can diff a run against a baseline to fail CI on regression.

6.1 Harness

backend/eval/harness.py:24-148 - AsyncEvalHarness

```
async def run_task(self, task_def) -> dict:
    acceptance_test = task_def["acceptance_test"]
    initial_state = { ... same 20 fields as pipeline.py, skip_tests=False,
                      skip_refactor=False, max_debug_attempts=self.max_debug_attempts }
    node_latencies = {}
    current_node_start = time.perf_counter()
    async for snapshot in agent_graph.astream(initial_state):
        node_end = time.perf_counter()
        for node_name, updated_state in snapshot.items():
            node_latencies[node_name] = node_latencies.get(node_name, 0.0) + (node_end - current_node_start)
            final_state.update(updated_state)
        current_node_start = time.perf_counter()

    final_code = final_state.get("final_code") or final_state.get("current_code", "")
    debug_attempts = final_state.get("debug_attempt", 0)
    agent_tests_passed = final_state.get("tests_passed", False)
    hit_max_debug = (debug_attempts >= self.max_debug_attempts) and (not agent_tests_passed)

    # held-out acceptance test - NOT the agent's own generated tests
    acceptance_res = await self.sandbox.run_tests(code=final_code, test_code=acceptance_test, language=language)
    passed = acceptance_res.get("success", False)

    is_sandbox_error = (acceptance_res.get("exit_code") == -1 or acceptance_res.get("timed_out", False)
                        or "Execution binary not found" in stderr_msg
                        or "Make sure Docker Desktop" in stderr_msg)
    pass_at_1 = passed and (debug_attempts == 0)
    pass_at_k = passed and (debug_attempts <= self.max_debug_attempts)
    return { task_id, language, difficulty, passed, agent_tests_passed, debug_attempts,
            hit_max_debug, pass_at_1, pass_at_k, total_latency, node_latencies,
            sandbox_error, fatal_error, sandbox_output, final_code, events }

async def run_dataset(self, tasks):
    results = []
    for task in tasks:
        # sequential - no gather
        results.append(await self.run_task(task))
    return results
```

- **Same graph, different verifier.** The agent's own `test_code` gates the internal debug loop; the harness then throws away trust in that and runs a *held-out* `acceptance_test` from `tasks.yaml` against `final_code`. "Passed" in the report always means "passed the held-out suite".
- **Per-node timing.** Wall-clock deltas between successive `astream` yields are attributed to the node named in each snapshot and summed (a node visited five times in the debug loop accumulates all five).
- **Sandbox-error vs code-wrong.** A failure is reclassified as infrastructure if the exit code is `-1`, it timed out, or the stderr contains the "binary not found" / "Docker Desktop" markers — so a missing compiler doesn't count against the model's pass rate.
- **Sequential, despite the name.** `run_dataset` is a plain `for` loop of `await` s. There is no `asyncio.gather`; an 18-task run is fully serialized. The multi-language coverage comes from the dataset (6 Python + 6 JS + 6 C++), not from parallel execution.

6.2 Every metric in plain English

backend/eval/metrics.py:13-69 - EvalMetrics.calculate_summary

```
passed_at_1_count = sum(1 for r in results if r.get("pass_at_1"))
passed_at_k_count = sum(1 for r in results if r.get("pass_at_k"))
give_up_count = sum(1 for r in results if r.get("hit_max_debug"))
sandbox_error_count = sum(1 for r in results if r.get("sandbox_error"))
repair_attempts_list = [r["debug_attempts"] for r in results if r.get("debug_attempts", 0) > 0]
avg_debug_repair = (sum(...) / len(...)) if repair_attempts_list else 0.0
```

Metric	Formula	Reads in plain English
Pass@1	<code>count(passed AND debug_attempts == 0) / total</code>	"Fraction of tasks the agent got right on the very first generation, with the held-out tests passing and <i>zero</i> trips through the debugger."
Pass@k	<code>count(passed AND debug_attempts <= max) / total</code>	"Fraction that eventually passed the held-out tests, whether first-shot or after up to 5 self-repair iterations." (Because <code>debug_attempts</code> can never exceed <code>max</code> , the <code><= max</code> clause is always true — in effect Pass@k here is just the overall pass rate.)
Give-Up Rate	<code>count(hit_max_debug) / total</code> , where <code>hit_max_debug = debug_attempts >= max AND not agent_tests_passed</code>	"Fraction of tasks where the agent burned all 5 debug attempts and still couldn't make its own tests pass — it exhausted the loop and surrendered."
Sandbox Failure Rate	<code>count(sandbox_error) / total</code>	"Fraction of tasks whose result was an infrastructure failure — missing compiler/interpreter, exit code -1, or timeout — not a wrong answer. Used to separate 'the model failed' from 'the environment failed'."
Avg Debug Iterations (<code>avg_debug_attempts_repair</code>)	mean of <code>debug_attempts</code> over <i>only</i> the tasks where <code>debug_attempts > 0</code>	"Among tasks that actually needed repair, how many debugger rounds did it take on average." Tasks solved first-shot are excluded so the number isn't diluted toward zero.
Avg Task Latency	mean of <code>total_latency</code> across all tasks	"Average end-to-end wall-clock seconds per task, including the held-out acceptance run."
Node Latency Breakdown (<code>avg_node_latencies</code>)	per node: <code>sum(node time across tasks) / (number of tasks that hit that node)</code>	"Average seconds spent in each graph node — <code>rag_context</code> , <code>planner</code> , <code>code_generator</code> , <code>sandbox_execute</code> , <code>test_generator</code> , <code>run_tests</code> , <code>debugger</code> , <code>refactor</code> , <code>finalise</code> . This is where you see the debugger dominating a slow run, or RAG dominating before the singleton fix."
by language / by difficulty	<code>_aggregate_subgroups</code> : same pass@1 / pass@k / avg latency, grouped	"The same headline numbers sliced by Python/JS/C++ and by easy/medium/hard, each with its own task count."

6.3 Comparator & regression gate

backend/eval/compare.py:24-97 – EvalComparator.compare_runs

```
def compare_runs(baseline_path, candidate_path, regression_threshold=0.05):
    base, cand = load(baseline_path), load(candidate_path)
    diff_metrics = { "pass_at_1": {baseline, candidate, delta}, "pass_at_k": {...},
                    "give_up_rate": {...}, "avg_latency": {...} }
    base_tasks = {t["task_id"]: t for t in base["tasks"]}
    cand_tasks = {t["task_id"]: t for t in cand["tasks"]}
    for tid, base_t in base_tasks.items():
        cand_t = cand_tasks.get(tid)
        if base_t["passed"] and not cand_t["passed"]: regressed_tasks.append(tid)
        elif not base_t["passed"] and cand_t["passed"]: improved_tasks.append(tid)
    p1_drop = diff_metrics["pass_at_1"]["delta"] < -regression_threshold
    pk_drop = diff_metrics["pass_at_k"]["delta"] < -regression_threshold
    has_regression = p1_drop or pk_drop or (len(regressed_tasks) > 0)
    return comparison_report, has_regression
```

A candidate run "regresses" if pass@1 or pass@k drops by more than 5 absolute points **or** if any individual task flips `passed` → `failed` (that second clause is strict — one flaky task fails the gate). `run_eval.py --compare baseline.json` prints the diff table and calls `sys.exit(1)` when `has_regression` is true; `compare.py` also has its own `__main__` CLI.

6.4 Dataset & CI

backend/eval/dataset/tasks.yaml – 18 tasks, structure

```
- id: py-easy-01
  task: "Write a Python function 'two_sum(...)' ..."
  language: "Python"
  framework: "standard library"
  difficulty: "easy"
  acceptance_test: |
    import pytest
    from solution import two_sum
    def test_two_sum_basic():
        assert two_sum([2,7,11,15], 9) == [0,1]
```

- 6 tasks per language × 3 languages = 18; each language has 2 easy / 2 medium / 2 hard. Content is classic interview algorithms (two-sum, LRU cache, merge-intervals, expression eval, topo-sort; flatten/title-case/deepClone/debounce/EventEmitter/AsyncTaskQueue; reverseWords/isAnagram/MinStack/productExceptSelf/LIS/Dijkstra).
- **Python** acceptance tests use `from solution import X`. **JavaScript** tests sandbox the solution with `vm.createContext` + `vm.runInContext` and pull the symbol from the VM globals/exports — robust to whether the model used `module.exports` or a bare global. **C++** tests forward-declare the signature and provide an `int main()` of `assert()`'s plus a success banner.

.github/workflows/eval.yml – CI job

```

on:
  pull_request: { paths: ['backend/**', '.github/workflows/eval.yml'] }
  workflow_dispatch:
steps:
  - setup-python 3.11 ; setup-node 20 ; apt-get install -y build-essential g++
  - pip install -r backend/requirements.txt pyyaml
  - working-directory: backend
    env: { GROQ_API_KEY: ${ secrets.GROQ_API_KEY },
          FORCE_LOCAL_SANDBOX: "false", ALLOW_LOCAL_SANDBOX: "true" }
  run: |
    BASELINE=eval/runs/baseline.json
    if [ -f "$BASELINE" ]; then
      python -m eval.run_eval --provider groq --model openai/gpt-oss-20b \
        --subset 5 --output-dir eval/runs --compare "$BASELINE" # exits 1 on regression
    else
      echo "::warning::No baseline.json - running without a regression gate."
      python -m eval.run_eval --provider groq --model openai/gpt-oss-20b \
        --subset 5 --output-dir eval/runs
    fi
  - # find newest runs/run_*.json, then actions/github-script posts/updates a PR comment

```

What the CI enforces, after revision fix #6 NOTE

- Runs a **5-task subset** (not the full 18) on every PR that touches `backend/**`, using Groq. Subset size is a cost/latency choice, not a limitation of the harness.
- The job now sets `ALLOW_LOCAL_SANDBOX="true"` deliberately — the GitHub runner has no Docker daemon, so execution uses the (now opt-in) local runner with `g++ / node / pytest` installed in the job. Without that env var the eval would abort by design.
- The step now runs `--compare eval/runs/baseline.json` **when that file exists**, so a pass@1 / pass@k drop > 5 pts or any task flipping pass→fail makes `run_eval.py` exit non-zero and fails the check. When no baseline is committed it emits a `::warning::` and posts the metrics comment only. Enabling the gate is a one-time "generate a good run, commit it as `baseline.json`" step (documented in the README).
- A latent bug was fixed alongside: the step now runs with `working-directory: backend`, so `python -m eval.run_eval` can actually import the `eval` package (previously it was invoked from the repo root where that import fails).
- The committed files in `backend/eval/runs/` are early local runs and all show **0% pass** — most failed on `GROQ_API_KEY is missing` or local-fallback import quirks, not model capability. They are historical artifacts; none is the baseline.

7. API layer & frontend streaming architecture

7.1 `/generate/stream` and the SSE contract

backend/api/routes.py:76-102 - `generate_stream`

```
class GenerateRequest(BaseModel):
    task: str
    language: str = "Python"
    framework: str = "standard library"
    provider: str = "ollama"
    model: str = "deepseek-coder:6.7b"
    skip_tests: bool = False
    skip_refactor: bool = False

@router.post("/generate/stream")
async def generate_stream(req: GenerateRequest):
    full_model = f"{req.provider}:{req.model}" if req.provider else req.model
    pipeline = AgentPipeline(model=full_model)

    async def event_stream():
        async for event in pipeline.run(task=req.task, language=req.language,
            framework=req.framework, skip_tests=req.skip_tests,
            skip_refactor=req.skip_refactor):
            yield f"data: {json.dumps(event)}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(event_stream(), media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

- **Framing.** Each graph event dict is serialized as one SSE record: `data: {json}\n\n`. A literal `data: [DONE]\n\n` sentinel terminates the stream. Event shape is always `{step, status, data}`.
- **Headers.** `media_type="text/event-stream"` is the SSE content type. `Cache-Control: no-cache` stops intermediaries caching the response. `X-Accel-Buffering: no` tells nginx (and other reverse proxies that honour it) not to buffer the response — without it, nginx would hold chunks until its buffer filled and the "real-time" terminal would arrive in bursts. The containerised deploy reinforces this in `docker/nginx.conf` with `proxy_buffering off; proxy_cache off; proxy_read_timeout 300s;` on the `/api/` location.
- **No manual chunk flushing.** Starlette's `StreamingResponse` sends each `yield`ed string as an HTTP chunk; there is no explicit `Transfer-Encoding` or `Connection` header set.
- **CORS.** `main.py` mounts `CORSMiddleware(allow_origins=["*"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"])`, and the router is mounted under `/api`.

7.2 Frontend: consuming the stream and driving the UI

frontend/src/useAgent.js:22-68 - the stream reader

```
const run = useCallback(async (config) => {
    if (abortRef.current) abortRef.current.abort()
    const controller = new AbortController()
    abortRef.current = controller
    setState({ ...INITIAL_STATE, status: 'running' })

    const res = await fetch(`${API}/generate/stream`, {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(config), signal: controller.signal,
    })
    const reader = res.body.getReader()
    const decoder = new TextDecoder()
    let buf = ''
    while (true) {
        const { done, value } = await reader.read()
        if (done) break
        buf += decoder.decode(value, { stream: true })
        const lines = buf.split('\n'); buf = lines.pop()
        for (const line of lines) {
            if (!line.startsWith('data: ')) continue
            const raw = line.slice(6).trim()
            if (raw === '[DONE]') { updateState({ status: 'success' }); continue }
            try { processEvent(JSON.parse(raw)) } catch {}
        }
    }
}, [])
```

It is a manual SSE reader, not `window.EventSource`. Because the request must be a `POST` with a JSON body (and `EventSource` only does GET), the hook uses `fetch + res.body.getReader() + TextDecoder` and re-implements SSE line framing: accumulate into `buf`, split on `\n`, keep the trailing partial line, and for every complete `data: ...` line parse the JSON. An `AbortController` stored in a ref lets the "Stop Agent" button (and a new run) cancel the in-flight stream. On a network error the hook sets `status: 'failed'`; an `AbortError` is swallowed.

frontend/src/useAgent.js:71-114 - `processEvent` → React state

```

const { step, status, data } = event
setState(s => {
  const newEvent = { step, status, data, timestamp, id: Date.now()+Math.random() }
  let patch = { events: [...s.events, newEvent] }
  if (step === 'CodeGenerator' && status === 'success' && data?.code) patch.currentCode = data.code
  if (step === 'TestGenerator' && status === 'success' && data?.test_code) patch.testCode = data.test_code
  if (step === 'Refactor' && status === 'success' && (data?.refactored_code || data?.code))
    patch.currentCode = data.refactored_code || data.code
  if (step === 'Pipeline' && status === 'success' && data?.final_code) {
    patch.currentCode = data.final_code
    patch.testCode = data.test_code || s.testCode || '' # keep suite visible
    patch.status = 'success' }
  if (step === 'Sandbox' && data?.stdout !== undefined)
    patch.execOutput = { stdout: data.stdout, stderr: data.stderr, success: data.success }
  if (step === 'CodeGenerator' && status === 'failed') {
    patch.status = 'failed'; patch.error = data?.error }
  return { ...s, ...patch }
})

```

UI element	File	Driven by
Pipeline progress bar — the 8 node chips (RAG → Plan → Generate → Execute → Tests → Debug → Refactor → Done) that light up idle/running/success/warning	components/PipelineBar.jsx	Reduces <code>state.events</code> to a <code>{stepKey: lastStatus}</code> map. Normalises the backend's <code>Debugger</code> step name to the bar's <code>DebugLoop</code> key so the Debug chip actually lights during repair.
Terminal log — timestamped, humanised event lines, auto-scrolled	components/TerminalLog.jsx	Maps each event through <code>eventMessage()</code> (a switch on <code>step / status / data</code>) to strings like "Running tests (attempt 2)...", "All tests passed after 3 attempt(s)", "Plan: 2 sub-task(s) — ...".
Code / Tests viewer — the syntax-highlighted panels behind the "Code" and "Tests" tabs, with a copy button	components/CodeViewer.jsx	<code>state.currentCode</code> / <code>state.testCode</code> . Highlighting is a hand-rolled tokeniser: HTML-escape → stash strings & comments as <code>___TOKEN_n___</code> placeholders → wrap a fixed Python/JS/C++ keyword set and <code>name()</code> call sites in <code></code> → restore placeholders → inject via <code>dangerouslySetInnerHTML</code> into a <code><pre></code> . The token-first ordering exists because doing strings after keywords corrupted the emitted markup.
Exec Output tab — stdout / stderr blocks, coloured by success	App.jsx	<code>state.execOutput</code> , updated on any <code>Sandbox</code> event carrying <code>stdout</code> .
Provider / Model / Language dropdowns	App.jsx	On mount, <code>fetch('http://localhost:8000/api/models')</code> → <code>providerInfo</code> . Provider select drives which model list renders (live Ollama list, or the hard-coded Groq/Gemini constants from <code>/models</code> , or the local <code>DEFAULT_MODELS</code> fallback). Selections go into the POST body of <code>run()</code> .
RAG knowledge-base counters + "Seed built-in docs" button	App.jsx, <code>useAgent.js</code>	<code>GET /api/rag/stats</code> on mount; the button POSTs <code>/api/rag/seed-docs</code> then refetches stats.
Tab auto-switching	App.jsx	<code>useEffect</code> : switch to "terminal" when a run starts; switch to "code" when <code>currentCode</code> arrives and the run is no longer running.

Minor config drift worth knowing before a live demo: `useAgent.js` and `App.jsx` hit an absolute `http://localhost:8000`, so the Vite dev proxy in `vite.config.js` (which also declares port 3050) is bypassed; the README, `start_project.bat`, and `docker/nginx.conf` all assume the frontend is on port 3000.

8. Gaps and weak points

This section now has two parts: **8A** — what was resolved in the post-audit revision — and **8B** — what was consciously left, each with the defence you would give in a review and the concrete path to close it.

8A. Resolved in this revision

Was	Now	Where
Error memory persisted a "fix" the instant the debugger LLM replied, before any re-test — the poisoning pattern behind <code>/api/rag/clear</code> .	The fix is staged in <code>state["pending_fix"]</code> and written only when a subsequent <code>run_tests</code> reports <code>tests_passed</code> . Unverified fixes never reach the store. (§3.3, §3.4)	<code>core/nodes.py</code> , <code>core/state.py</code>
No Docker → generated code ran on the host automatically, with only an unread <code>stderr</code> warning.	<code>LocalSandboxDisabledError</code> is raised unless <code>ALLOW_LOCAL_SANDBOX</code> / <code>FORCE_LOCAL_SANDBOX</code> is set; the graph catches it, sets <code>error</code> , and <code>should_skip_tests</code> aborts to <code>finalise</code> . (§2.2)	<code>core/sandbox.py</code> , <code>core/nodes.py</code> , <code>core/graph.py</code>
<code>/api/rag/index-project</code> walked any caller-supplied path — a file-disclosure primitive.	<code>_safe_index_path</code> resolves the path and rejects anything outside the project root. (§4.2)	<code>api/routes.py</code>
<code>langgraph</code> was a hard import absent from <code>requirements.txt</code> ; <code>langchain</code> pin was stale (<code>0.2.0</code> vs installed <code>1.3.2</code>).	<code>langgraph==1.2.2</code> pinned; <code>langchain==1.3.2</code> corrected.	<code>backend/requirements.txt</code>
<code>debug_context</code> declared but never used; <code>max_debug_attempts</code> in state ignored by the graph, so the eval <code>--max-debug</code> flag didn't change the loop.	<code>debug_context</code> removed; <code>after_tests_run</code> reads <code>state["max_debug_attempts"]</code> with the module constant as fallback. (§1.2, §1.3)	<code>core/state.py</code> , <code>core/graph.py</code>
CI never ran <code>--compare</code> , so the regression gate was unreachable; the eval step also had a working-directory import bug.	CI runs <code>--compare eval/runs/baseline.json</code> when present (fails on regression), sets <code>ALLOW_LOCAL_SANDBOX=true</code> for the daemon-less runner, and runs from <code>backend/</code> . Committing a good run as <code>baseline.json</code> arms the gate. (§6.4)	<code>.github/workflows/eval.yml</code>

8B. Deferred — with defence

8B.1 No LangGraph checkpointing across process restarts

`build_graph()` calls `g.compile()` with no `checkpointers`; there is no `thread_id` and no resume path. With `uvicorn ... reload=True`, a file save kills in-flight runs; a deploy or crash does the same in production. The SSE socket drops and the user re-submits.

Defence. The delivered scope is a single-user local assistant driven by one synchronous browser session. In that context the cost of a lost run is one re-submit, and revision fix #1 removed the only way a dead run could leave *corrupt* state (unverified error-memory rows) — a verified fix that was already committed earlier in the run is a correct row, just an orphaned one. Building half a durability layer (a checkpointer with no resume endpoint, or resume with no idempotency on the ChromaDB writes) would be worse than the honest current behaviour.

Path to close. Compile with `MemorySaver` (in-process resume) or `SQLiteSaver` / `PostgresSaver` (cross-restart); mint a `run_id`, return it in the first SSE event; add `GET /generate/stream/{run_id}` that replays the persisted event log then re-attaches via `astream(..., config={"configurable": {"thread_id": run_id}})`. Make `record_successful_fix` idempotent by keying the document id on a hash of `(task, error, fixed_code)` instead of a random UUID, so a replayed debug step can't double-write. This is a boundary change (~15–20 lines at `compile()` and the endpoint); node logic is untouched.

8B.2 Auth, rate limiting, CORS

Every `/api` route is unauthenticated; CORS is `allow_origins=["*"]` with `allow_credentials=True` (a contradictory pair); there are no per-token quotas or request-size caps. `/api/generate/stream` is an open way to spend the operator's LLM budget; `/api/rag/clear` is an open way to wipe the vector stores. (The sharpest instance — arbitrary-path indexing — was closed in 8A.)

Defence. The service binds for local use and is documented as such; the frontend talks to `localhost:8000`. No credentials, PII, or multi-tenant data are involved, and the LLM keys live in the operator's own `.env`. Adding auth now would be securing a door in a building with no walls — the deployment story (who hosts it, for whom) has to exist first.

Path to close. Before any shared/hosted deployment: an API-key or session dependency on the router; `slowapi` (or a reverse-proxy limit) on `/generate/*`; a small allow-list of origins replacing `*` and dropping `allow_credentials`; a body-size limit on `/generate/stream`; move `/api/rag/clear` behind an admin scope.

8B.3 Single-file scope — no multi-file generation or repo-wide refactor

The pipeline writes exactly one `solution.<ext>` and one test file. The `### FILE: <name> ###` separator in `CODE_GENERATOR_SYSTEM` is never parsed; the planner's per-subtask `files: [...]` is displayed but never drives generation; refactor operates on the single `current_code` string.

Defence. This is a deliberate scope line, not an oversight. The hard, novel part of the project — a verified self-repair loop with execution feedback and error memory — is fully exercised by single-artifact tasks, which is also what the 18-task benchmark measures. Multi-file support multiplies the surface (dependency ordering, partial-application, cross-file test wiring) without changing the thing the project is demonstrating.

Path to close. Replace `current_code: str` with `files: dict[str, str]` in state; a writer node materialises the map into the sandbox workdir; the planner's `files` field selects which entries each generation/debug step targets; the sandbox's single-file assumption (`solution.<ext>`) becomes a manifest. The debug loop and RAG layers are unaffected.

8B.4 Remaining state-contract slack

`plan` is produced by `node_planner` but only surfaced in its event — no node consumes it. `events` has no LangGraph reducer, so every node hand-concatenates `state["events"] + [...]`; a future *parallel* node would clobber history. A non-fatal failure of the initial `run_code` (a crash, wrong output) still proceeds to test generation — only a fatal `error` now aborts there.

Defence. None of these affect the happy path. `plan` is retained as genuine produced data (removing it risks a LangGraph "unknown channel" error in `node_planner`'s return for zero benefit). The graph is strictly linear with one cycle — no parallel fan-out exists — so the missing `events` reducer is latent, not active. Letting a crashing first run continue to test generation is intentional: the generated test suite frequently pins down *why* it crashed better than the bare run does, and that output feeds the debugger.

Path to close. Annotate `events` as `Annotated[List, operator.add]` the moment any parallel node is added; wire `plan` into generation as part of 8B.3; if desired, add an `exec_success` branch to `should_skip_tests` that routes a hard crash straight to the debugger instead of through test generation.

8B.5 Evaluation harness

`run_dataset` is a sequential `for` loop of `await s` — no `asyncio.gather` — so an 18-task run is fully serialized. `pass_at_k`'s second clause (`debug_attempts <= max`) is always true, so the metric equals the overall pass rate. The committed `runs/*.json` are all 0% (missing key / early bugs) and none is a baseline. (CI `--compare` wiring and the working-directory bug were fixed in 8A.)

Defence. Serial execution is a correctness choice: parallel tasks would contend on the shared ChromaDB error-memory collection and on Docker/CPU, making latency numbers and any cross-task memory effects non-reproducible — the eval prioritises stable measurement over wall-clock. `pass_at_k` is reported honestly next to `pass_at_1`; with a fixed loop bound it is by definition "solved within k", and the give-up rate captures the complement.

Path to close. Add an opt-in `--concurrency N` that `gather`s tasks with a semaphore and a per-worker temp DB; make `pass_at_k` meaningful by sampling `k` independent generations per task and reporting "any passed"; generate and commit a real `baseline.json` to arm the CI gate (README documents the one-liner).

8B.6 Provider abstraction rough edges

Groq/Gemini "configured" is pure env-var presence — an invalid key looks healthy until a run dies in `node_code_generator`. Model lists are hard-coded constants. A fresh `httpx.AsyncClient` is built per call and per retry. Ollama has no 429/5xx retry. The Gemini key rides in the URL query string.

Defence. A live "is this key valid" probe costs a real API call (and quota) on every page load; deferring the failure to first use is a reasonable trade for a local tool, and the error that surfaces (`GROQ_API_KEY is missing` / an HTTP status) is specific. Per-call clients are simple and, at one LLM request every few seconds, connection-pool reuse saves nothing measurable. Ollama is a localhost call with no rate limit to hit.

Path to close. Cache a validated-key flag with a short TTL and a cheap models-list call; pull provider model lists from their APIs where available; hoist a module-level `httpx.AsyncClient` with limits; move the Gemini key to the `x-goog-api-key` header; give Ollama the same bounded retry as the cloud providers.

8B.7 Frontend

`CodeViewer` renders model output via `dangerouslySetInnerHTML` after a hand-rolled escape-then-tokenise pass. The API base is a hard-coded `http://localhost:8000`, and the dev port drifts (3000 / 3050) across README, Vite config, batch script, and nginx.

Defence. The highlighter HTML-escapes `& < >` as step 1, before any span is introduced, and only ever wraps a fixed keyword set and `name()` call sites — the token-store rewrite exists specifically to stop later passes matching earlier markup. It is a contained, reviewed function over output the same user just requested, not third-party content. The port drift is cosmetic: the frontend uses an absolute API URL, so the Vite proxy (and its port) is never on the path.

Path to close. Swap the custom highlighter for a vetted library that emits DOM nodes rather than an HTML string (removes the `dangerouslySetInnerHTML` entirely); read the API base from `import.meta.env.VITE_API_BASE`; reconcile the dev port to one value across all four files.

8B.8 Observability

No structured logging, no tracing, no run ids, no metrics endpoint. The SSE event stream is the only record of a run and is gone when the socket closes.

Defence. For local single-user use the browser is the observability surface — the terminal log shows every node, every debug attempt, every test transcript in real time, which is more than most tools expose. There is no fleet of runs to aggregate.

Path to close. Pairs naturally with 8B.1: once runs have a `run_id` and a checkpoint, persist the event log per run, add a `GET /runs/{id}`, emit `structlog` JSON lines keyed by `run_id / node`, and expose a `/metrics` counter set (runs, pass rate, mean debug attempts, node latencies) — the eval's `EvalMetrics` already computes exactly these.

If I were reviewing this as a principal engineer, the first thing I'd still ask the author to defend

"Walk me through what happens to an in-flight task when the process restarts." The revision closed the part of this that was a bug — a dead run can no longer poison error memory with an unverified fix. What remains is a deliberate scope decision: no checkpoint, the run tied to one HTTP response. I would want the author to (a) state that boundary out loud rather than discover it live, (b) show the ~15-line `SQLiteSaver + run_id` resume design in 8B.1 as evidence they know how to cross it, and (c) name the one subtlety — making `record_successful_fix` idempotent so a replayed debug step doesn't double-write. An author who volunteers all three has a stronger answer than one whose project simply happens to have a checkpoint.

Close second: "your Dockerised Python test runner can't install pytest offline — when has the container path actually run green for Python?" The honest answer is "it hasn't; Python tests run through the local runner." The fix is a five-line `Dockerfile.sandbox-python` (§2.3), and I would expect it done before the sandboxing claim is made unqualified.

9. One-page mental model — the request lifecycle

Everything you need to narrate CodeAgent Pro from memory. One task query, start to finish.

The 60-second version

A React SPA POSTs a task to a FastAPI endpoint that streams Server-Sent Events. The endpoint runs a **LangGraph state machine** (compiled once, in `core/graph.py`) whose nodes are: retrieve context from a local ChromaDB, plan, generate code with an LLM, run it in a Docker sandbox, generate a test suite, run the tests, and — if they fail — loop through a **debugger** up to 5 times, consulting a growing "error memory" of past fixes, before a final refactor pass. Each node appends an event to shared state; a thin adapter (`core/pipeline.py`) diffs that event log after every node and streams the new entries to the browser, which paints a progress bar, a terminal log, and code panels in real time. Nothing is persisted — the run lives and dies with the HTTP response; only ChromaDB writes outlast it. The same graph is reused offline by `eval/` to benchmark models against 18 held-out tasks.

The full path, with exact names

- Browser.** `App.jsx` `handleRun()` → `useAgent.js` `run(config)` → `fetch('POST http://localhost:8000/api/generate/stream')` with body `{task, language, framework, provider, model, skip_tests, skip_refactor}`. The response body is read with `res.body.getReader()` + `TextDecoder`; SSE `data: ...` records are framed by hand.
- API.** `backend/main.py` (FastAPI, CORS *, router under `/api`) → `backend/api/routes.py:generate_stream()`. Builds `full_model = f"{provider}:{model}"`, constructs `AgentPipeline(model=full_model)`, returns a `StreamingResponse(event_stream(), media_type="text/event-stream", headers={"X-Accel-Buffering": "no", "Cache-Control": "no-cache"})`. `event_stream()` wraps each pipeline event as `data: {json}` and ends with `data: [DONE]`.
- Adapter.** `backend/core/pipeline.py:AgentPipeline.run()` builds the initial `PipelineState` dict (`core/state.py`) and iterates `agent_graph.astream(initial_state)` (the compiled singleton from `core/graph.py:build_graph()`). For each `{node: state}` snapshot it slices `state["events"]` past a high-water mark and yields the new events. A crash → one synthetic `Pipeline/failed` event; always ends with `Done/success`.
- Graph walk** (nodes in `backend/core/nodes.py`):
 - `node_rag_context` → `RAGManager.get_generation_context(task, language)` (`rag/rag_manager.py`) → top-3 from the `docs` collection + top-3 from the `codebase` collection (`rag/base_store.py`, ChromaDB, `all-MiniLM-L6-v2`, cosine). Writes `rag_context`.
 - `node_planner` → `PlannerAgent.run()` (`agents/planner.py`) → LLM via `core/llm_client.py:OllamaClient.generate()` (provider chosen by the `groq: / gemini: / bare` prefix) → JSON sub-tasks. Advisory only.
 - `node_code_generator` → `CodeGeneratorAgent.run()` (`agents/code_generator.py`); `rag_context` is prepended as `RELEVANT CONTEXT:`; 3 internal retries; `_clean_code` extracts the largest fenced block. Writes `current_code`, or sets `error`. Router `should_abort: empty/error` → `finalise`.
 - `node_sandbox_execute` → `ExecutionSandbox.run_code()` (`core/sandbox.py`): Docker `python:3.11-slim / node:20-alpine / gcc:latest` with `--network none --memory 256m`, read-only mount. No Docker and no `ALLOW_LOCAL_SANDBOX` → `LocalSandboxDisabledError` → `error` set. Router `should_skip_tests` aborts to `finalise` on `error` or `skip_tests`.
 - `node_test_generator` → `TestGeneratorAgent.run()` (`agents/test_generator.py`), 3 retries. Writes `test_code` or sets `error`. Router `should_abort_after_test_gen`.
 - `node_run_tests` → `ExecutionSandbox.run_tests(): pytest -v --tb=short / node --test / g++ + assert-main, --memory 512m, 60 s`. "collected 0 items" / "no tests ran" forces `tests_passed=False`. Writes `tests_passed`, `last_error = stdout+"\n"+stderr`. If `tests_passed` and a `pending_fix` is staged, commit it to error memory. Router `after_tests_run: pass` → `refactor` (or `finalise` if `skip_refactor`); `debug_attempt >= max_debug_attempts` (default 5) → `finalise`; else → `debugger`.
 - `node_debugger` → `RAGManager.get_debug_context(last_error)` (top-1 from `error_memory`, similarity > 0.65) is appended to the error text → `DebuggerAgent.run(current_code, augmented_error, attempt)` (`agents/debugger.py`) → new `current_code`. The candidate (`task, language, error, broken_code, fixed_code`) is `staged` in `state["pending_fix"]`, not yet written. `debug_attempt += 1`. Unconditional edge back to `node_run_tests` — this is the cycle; on the next visit, if the run passes, `node_run_tests` commits `pending_fix` to the `error_memory` ChromaDB collection.
 - `node_refactor` → `RefactorAgent.run(current_code, language)` (`agents/refactor.py`), language-locked. Writes `final_code`.
 - `node_finalise` → emits `Pipeline/success` with `final_code` + `test_code`.
- UI update.** Every node appends `{step, status, data}` to `state["events"]`; the adapter streams them; `useAgent.processEvent` folds them into React state → `PipelineBar.jsx` (node chips), `TerminalLog.jsx` (humanised lines), `CodeViewer.jsx` (custom highlighter, Code/Tests tabs), and the Exec Output tab. `data: [DONE]` → `status: 'success'`.
- Persistence.** None for the run. The graph is compiled with no checkpointing; state is in-memory + the open socket. Only ChromaDB side effects (seeded docs, indexed code, recorded fixes under `backend/data/chromadb/`) survive a restart.

The offline branch (same engine)

`python -m eval.run_eval --provider ... --model ...` → `eval/run_eval.py` loads `eval/dataset/tasks.yaml` (18 tasks) → `eval/harness.py:AsyncEvalHarness.run_task()` runs the same `agent_graph.astream`, then executes the task's `held-out` `acceptance_test` in `ExecutionSandbox` → `eval/metrics.py:EvalMetrics.calculate_summary()` computes Pass@1, Pass@k, Give-Up Rate, Sandbox Failure Rate, Avg Debug Iterations, per-node latency, and language/difficulty splits → `eval/report.py:EvalReport.save_run()` writes `eval/runs/run_{ts}_{model}.json` → optional `eval/compare.py:EvalComparator.compare_runs()` diffs against a baseline and exits non-zero on regression. `.github/workflows/eval.yml` runs a 5-task subset on PRs touching `backend/**` and comments the metrics table.

End of audit. Code blocks are quoted from the repository at HEAD `ce114fa` plus the post-audit revision described in the Revision note; file paths are given in each block header so claims can be checked directly against source. Sections tagged **FIXED** reflect changes now present in the working tree.